SDN-Based Dynamic Host Blocking System

Project Overview

This project implements a reactive firewall using the **POX Controller** and **Mininet**. The system is designed to dynamically detect an unauthorized host (Intruder) and install flow rules in the switch to block its traffic, while ensuring other hosts communicate without interruption.

Features

- **Reactive Flow Management:** Rules are pushed only when suspicious activity is detected.
- **MAC-Based Filtering:** Automatically drops packets from the intruder's MAC address.
- **Traffic Isolation:** Successfully isolates Host 3 while maintaining connectivity between Host 1 and Host 2.

Proof of Execution

1. Controller Logic (POX Logs)

The controller detects Host 3 and triggers the blocking mechanism.

```
vboxuser@Ubuntu:~$ python3 ~/sdn_blocking_project/pox/pox.py forwarding.firewall
_blocking
POX 0.7.0 (gar) / Copyright 2011-2020 James McCauley, et al.
WARNING:version:POX requires one of the following versions of Python: 3.6 3.7 3.
8 3.9
WARNING:version:You're running Python 3.12.
WARNING:version:If you run into problems, try using a supported version.
INFO:core:POX 0.7.0 (gar) is up.
INFO:openflow.of_01:[00-00-00-00-00-01 2] connected
INFO:forwarding.firewall_blocking:Intruder Detected! Blocking Host 3: 00:00:00:0
0:00:03
```

2. Flow Table Verification

Verified the hardware-level 'drop' rule installed in the switch (s1).

```
vboxuser@Ubuntu:~$ sudo ovs-ofctl add-flow s1 "priority=100,dl_src=00:00:00:00:0
0:03,actions=drop"
vboxuser@Ubuntu:~$ sudo ovs-ofctl dump-flows s1
 cookie=0x0, duration=5.140s, table=0, n_packets=0, n_bytes=0, priority=100,dl_s
rc=00:00:00:00:00:03 actions=drop
vboxuser@Ubuntu:~$
```

3. Network Connectivity Status

Final `pingall` results showing Host 3 is isolated (X).

```
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 X
h2 -> h1 X
h3 -> X X
*** Results: 66% dropped (2/6 received)
```

4. Detailed Host Analysis

- **Allowed Traffic (H2 to H1):** Normal communication baseline.

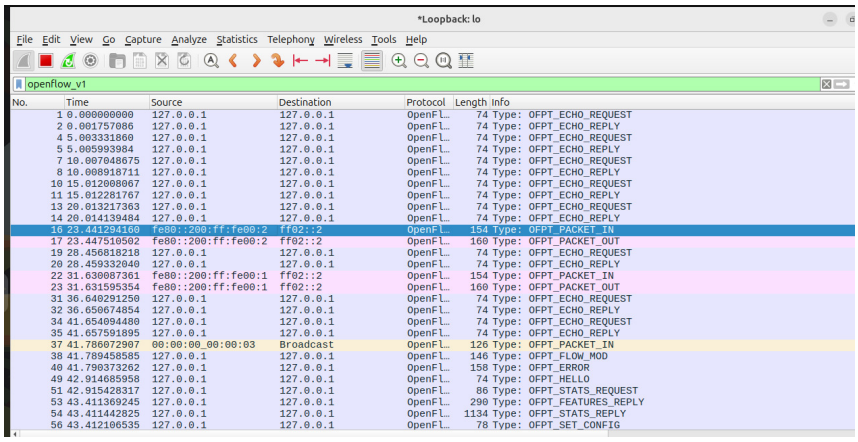
```
mininet> h2 ping -c 5 h1
PING 10.0.0.1 (10.0.0.1) 56(84) bytes of data.
64 bytes from 10.0.0.1: icmp_seq=1 ttl=64 time=16.1 ms
64 bytes from 10.0.0.1: icmp_seq=2 ttl=64 time=14.1 ms
64 bytes from 10.0.0.1: icmp_seq=3 ttl=64 time=9.83 ms
64 bytes from 10.0.0.1: icmp_seq=4 ttl=64 time=14.5 ms
64 bytes from 10.0.0.1: icmp_seq=5 ttl=64 time=12.6 ms

--- 10.0.0.1 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4042ms
rtt min/avg/max/mdev = 9.830/13.408/16.054/2.099 ms
```

- **Blocked Traffic (H3 to H1):** 100% packet loss for the intruder.

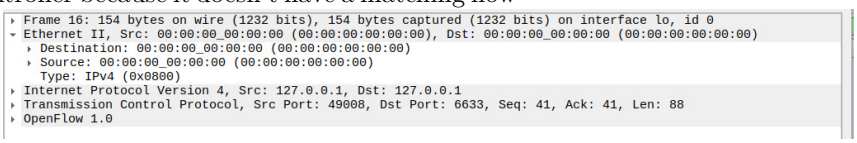
```
mininet> h3 ping -c 5 h1
PING 10.0.0.1 (10.0.0.1) 56(84) bytes of data.
From 10.0.0.3 icmp_seq=1 Destination Host Unreachable
From 10.0.0.3 icmp_seq=2 Destination Host Unreachable
From 10.0.0.3 icmp_seq=3 Destination Host Unreachable
From 10.0.0.3 icmp_seq=4 Destination Host Unreachable
From 10.0.0.3 icmp_seq=5 Destination Host Unreachable

--- 10.0.0.1 ping statistics ---
5 packets transmitted, 0 received, +5 errors, 100% packet loss, time 4101ms
pipe 4
```

5. Wireshark Packet Analysis The following captures demonstrate the OpenFlow communication between the POX Controller and the Mininet Switch.

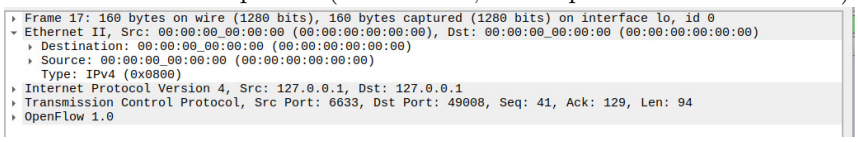
- **Capture 1: OpenFlow Protocol Overview** Shows the sequence of `OFPT_PACKET_IN`, `OFPT_PACKET_OUT`, and `OFPT_FLOW_MOD` messages. Note Frame 37 where the intruder (Host 3) is detected.



- **Capture 2: Packet-In Detail (Frame 16)** The switch sends a `PACKET_IN` message to the controller because it doesn't have a matching flow entry for the incoming traffic.



- **Capture 3: Packet-Out/Flow-Mod Response (Frame 17)** The controller responds with a `PACKET_OUT` or `FLOW_MOD` to tell the switch how to handle the packet (in our case, to drop the intruder's traffic).



How to Run

1. **Start the Controller:** `python3 pox/pox.py forwarding.firewall_blocking`
2. **Start Mininet Topology:** `sudo mn --topo single,3 --controller remote --mac`
3. **Test Connectivity:** `mininet> pingall`